



COMMAND PATTERN

1. Mục tiêu bài học

- Hiểu bản chất Command Pattern (không nhầm với Service/Method)
- Nhận diện code xử lý hành động bị if–else/switch-case bùng nổ
- Đóng gói hành vi (PlaceOrder, CancelOrder...) thành Command object
- Áp dụng Command cho:
 - Undo / Redo
 - Queue / Logging / Audit
- Liên hệ với Strategy, Observer & Event-driven architecture

2. Đặt vấn đề – “Hành động” bị cứng

Trong hệ thống Order, ta có rất nhiều action:

- Place Order, Cancel Order, Confirm Payment, Ship Order...

🔗 Nếu xử lý theo cách truyền thống:

```
if (action == "PLACE") { ... }
```

```
else if (action == "CANCEL") { ... }
```

```
else if (action == "SHIP") { ... }
```

✗ Vấn đề:

- Vi phạm OCP (Sửa code cũ khi thêm action mới)
- Business logic bị trộn lẫn trong Controller
- Khó log / undo / queue từng hành động riêng lẻ

3. Ý tưởng & Định nghĩa

👉 Ý tưởng cốt lõi:

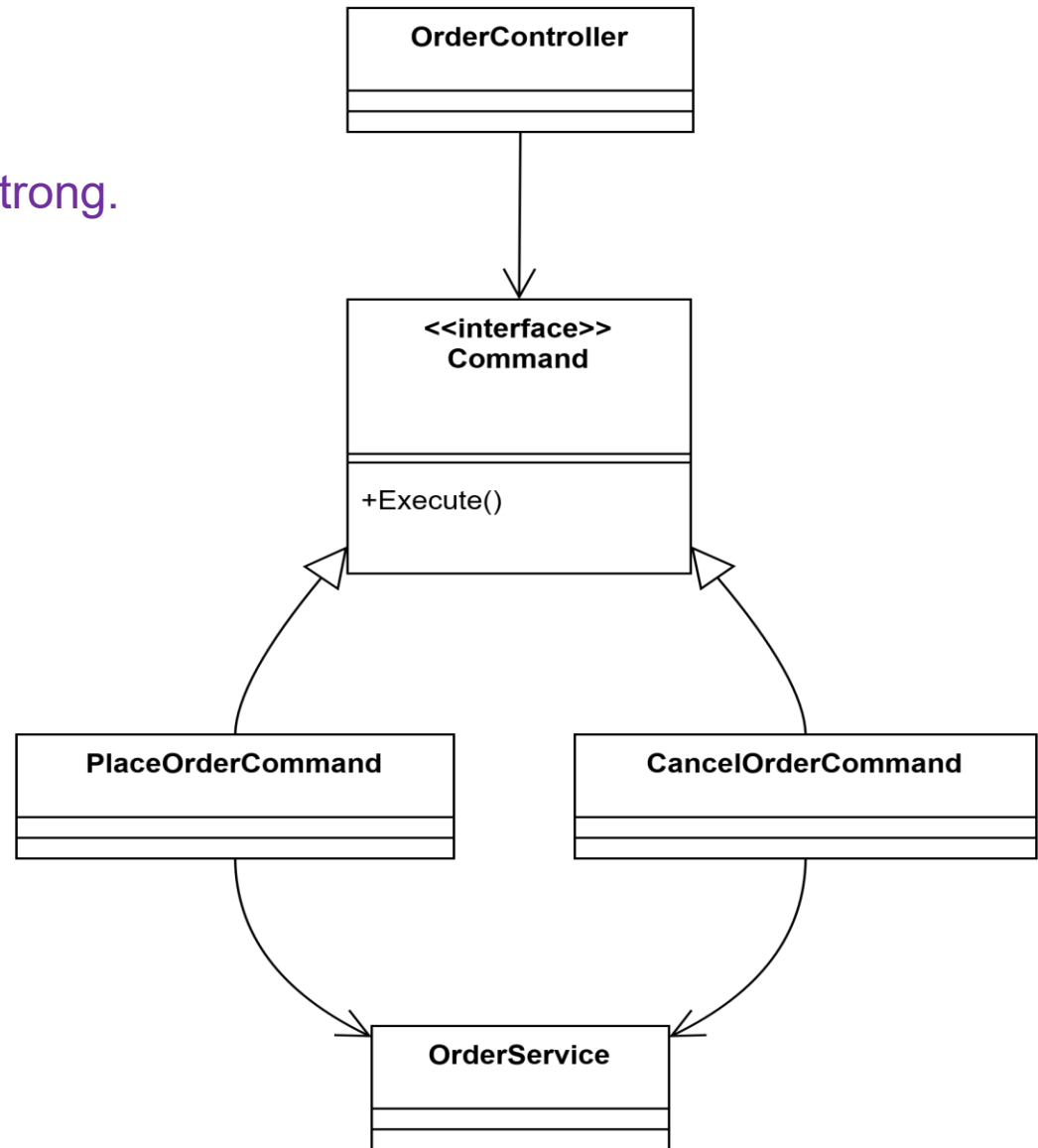
- Biến mỗi hành động (Request) thành một Object độc lập.
- Thay vì: “Gọi method để làm gì đó”
- Ta có: “Đưa cho hệ thống một Command để thực thi”

📌 Định nghĩa (GoF):

- Command Pattern đóng gói một request thành một object, cho phép tham số hóa client, hỗ trợ undo/redo/logging.

4. UML – Command Pattern

📌 Controller chỉ biết Command chung, không biết logic chi tiết bên trong.



5. Thiết kế ban đầu – VI PHẠM COMMAND

✗ Controller ôm quá nhiều quyết định, vi phạm OCP.

```
public class OrderController {  
    private readonly OrderService _service;  
  
    public void Handle(string action, Order order) {  
        if (action == "PLACE")  
            _service.PlaceOrder(order);  
        else if (action == "CANCEL")  
            _service.CancelOrder(order.Id);  
        // Action mới -> Lại phải sửa ở đây! ✗  
    }  
}
```

6. Refactor – Định nghĩa Command

1. Interface

```
public interface IOrderCommand {  
    void Execute();  
}
```

2. Concrete Command

```
public class PlaceOrderCommand : IOrderCommand {  
    private readonly OrderService _service;  
    private readonly Order _order;  
    public PlaceOrderCommand(OrderService service, Order order) {  
        _service = service;  
        _order = order;  
    }  
    public void Execute() => _service.PlaceOrder(_order);  
}
```

7. OrderService (Receiver)

 OrderService chỉ giữ nghiệp vụ cốt lõi, không biết ai gọi mình.

```
public class OrderService {  
    // Đây là "Receiver" - Người thực thi thực sự  
    public void PlaceOrder(Order order) {  
        Console.WriteLine("Executing logic: Place Order...");  
    }  
  
    public void CancelOrder(int orderId) {  
        Console.WriteLine("Executing logic: Cancel Order...");  
    }  
}
```


8. Invoker (Controller)

✚ Controller rất mỏng, chỉ biết Execute().

```
public class OrderController {  
    // Controller không còn if-else  
    public void ExecuteCommand(IOrderCommand command) {  
        // Có thể log action ở đây  
        Console.WriteLine($"Start command:  
{command.GetType().Name}");  
  
        command.Execute();  
  
        // Có thể push vào queue hoặc history  
    }  
}
```

9. Cách sử dụng (Client)

```
var service = new OrderService();  
var controller = new OrderController();  
  
// Tạo command (đóng gói request)  
IOrderCommand cmd = new PlaceOrderCommand(service, new  
Order());  
  
// Thực thi  
controller.ExecuteCommand(cmd);  
  
// Có thể lưu 'cmd' vào list để Undo sau này!
```

10. Lợi ích: Before vs After

Tiêu chí	Trước (If-else)	Sau (Command Pattern)
Cấu trúc	Logic trộn lẫn	Mỗi action là 1 class
Controller	Phình to (Bloaters)	Mỏng, gọn nhẹ
Mở rộng	Sửa code cũ (vi phạm OCP)	Thêm class Command mới
Tính năng	Khó Log/Undo	Dễ dàng Log/Undo/Queue

11. Command + Undo + Queue

📌 Mở rộng 1: Undo / Redo

```
interface IUndoableCommand {  
    void Execute();  
    void Undo();  
}
```

=> Áp dụng: Cancel Order, Rollback Payment.

📌 Mở rộng 2: Queue / Async

Command object có thể được serialize và đẩy vào Message Queue (RabbitMQ).

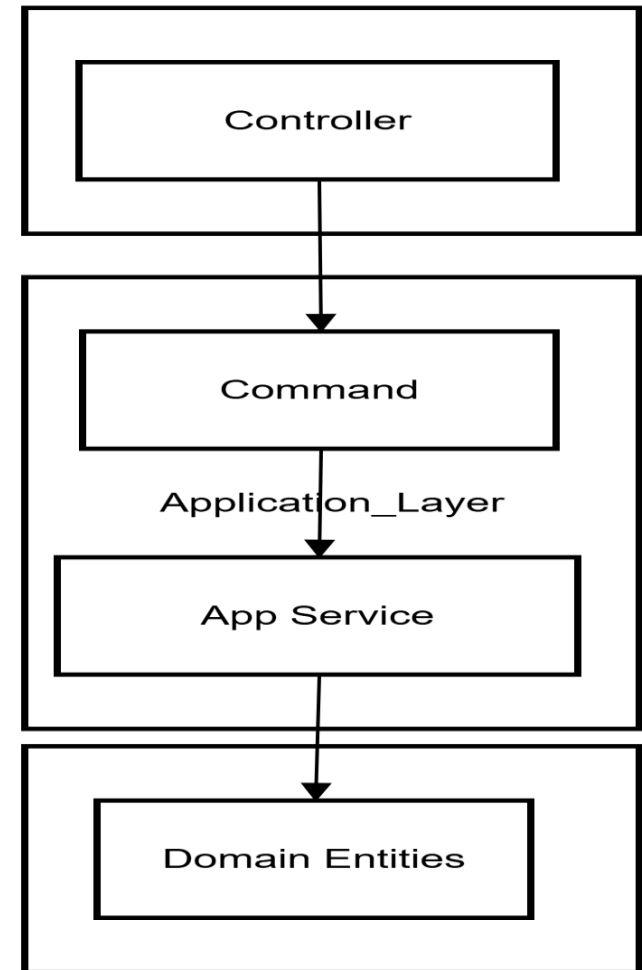
=> Nền tảng của CQRS và System bất đồng bộ.

12. So sánh: Command vs Strategy

Tiêu chí	Command	Strategy
Mục đích	Đóng gói hành động (Action)	Đóng gói thuật toán (Algo)
Thời điểm	Khi cần lưu trữ/hoãn lại request	Khi cần thay đổi cách làm ngay
Đối tượng	Request (What to do)	Behavior (How to do)

13. Command trong Clean Architecture

📌 Command nằm ở Application Layer
(Use Cases). Không nằm trong Domain.



14. Bài tập trên lớp



Bài 1:

Viết thêm `ShipOrderCommand` và `RefundOrderCommand`.



Bài 2:

Refactor một đoạn code Controller cũ đang dùng switch-case sang Command Pattern.



Bài 3:

Cài đặt tính năng "Undo" cho `PlaceOrderCommand` (Gợi ý: Xóa Order vừa tạo).

15. Kết luận

“Đừng hỏi hệ thống làm gì. Hãy đưa cho nó một Command.”

“Command giúp hệ thống mở rộng hành động mà không làm vỡ kiến trúc.”